

Documentation for Running the Scaling Code

Code description

The code solves Boussinesq equations in N^3 grid points using MPI parallelisation. The time step size is fixed at $dt = 10^{-3}$. The code is executed for a total simulation time T , so that the total number of time steps is T/dt . A detailed step-by-step procedure for running the code is given below.

The total walltime taken by the code will be saved in a file named **Total Time_taken.txt**.

Steps to run the code

- Install the required Python packages: **jax**, **sympy**, and **tqdm**.
- To change the spatial resolution of the simulation, vary the variable **N** in the code. For example, **N = 256** corresponds to a 256^3 grid.
- The time step is fixed at $dt = 10^{-3}$. To change the total simulation time, vary the variable **T**. The total number of RK4 steps is therefore **steps = T/dt**.
- To control how frequently simulation data are written to disk, vary the variable **save_every**. Setting **save_every = 1** means that data are saved at every time step. Setting **save_every = 0** disables saving completely and is used for the no-saving benchmark.
- **IMPORTANT: Perform the following four tests.** Keep **dt**, **T**, and all other simulation parameters identical between the tests. Only change **N** and **save_every** as specified below.
- **Test 1 — N = 256, no saving:** Set **N = 256** and **save_every = 0**. Run the simulation and record the final simulation performance.
- **Test 2 — N = 256, saving at every timestep:** Set **N = 256** and **save_every = 1**. Data will be written to disk after every time step. Run the simulation and record the final simulation performance.
- **Test 3 — N = 384, no saving:** Set **N = 384** and **save_every = 0**. Run the simulation and record the final simulation performance.
- **Test 4 — N = 384, saving at every timestep:** Set **N = 384** and **save_every = 1**. Data will be written to disk after every time step. Run the simulation and record the final simulation performance.
- **For each of the four tests, quote the performance numbers printed at the end of the simulation.** In particular, report the **Total RK4 steps**, **Total wall-clock time**, **Average iterations/s**, **Average time/step**, and **Saving interval**.
- The output at the end of the code will look similar to:

```
=====
FINAL SIMULATION PERFORMANCE
=====
Total RK4 steps : 100
Total wall-clock time : 36.160360 s
Average iterations/s : 2.765 iter/s
Average time/step : 361.604 ms
Saving interval : every 1 steps
=====
```